

Lab 7 – Convolution-II*

EE252L: Signals and Systems Lab

Dr. Basit Memon and Ms. Hira Mustafa

Name: _____

Objectives

By the end of this lab, you will be able to get familiarize with 2-D convolution and its application in bar-code detection system.

Instructions: After completion of each task get it marked by sharing your results and code with any of the RA, your code must be self-explanatory i.e. use proper comments. *Attach rubrics at the end of your report.*

1 Filtering

In this lab, we'll continue to experiment with convolution for the purposes of filtering. Recall that, filtering is the process of modifying or removing some features of a signal and a filter is a system that accomplishes this purpose. For example, removing the ambient noise from an audio signal so that you can hear the speech signal better can be accomplished by designing an appropriate filter. We have been focusing on filters that are linear and time-invariant. Remember that an LTI system can be completely described by its impulse response, which means that designing a filter corresponds to describing the impulse response, h , of the filter. The action of filtering is convolving our input signal with this designed impulse response of the filter, i.e.

$$\begin{aligned} y[n] &= x[n] * h[n] && \text{in discrete-time case,} \\ y(t) &= x(t) * h(t) && \text{in continuous-time case.} \end{aligned}$$

The values of the impulse response are also called the coefficients of the filter. In discrete-time case, filters are divided into two categories based on the length of their impulse responses – *finite impulse response (FIR) filters* and *infinite impulse response (IIR) filters*. This lab will introduce working with images in MATLAB and applying some FIR filters to images using convolution.

*Adapted from DSP First, 2e

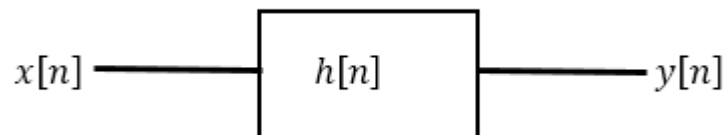


Figure 1: Filtering via convolution

2 2D-Convolution

The filters introduced in the previous lab were 1D filters, while an image is a 2D signal. We can still apply those 1D filters to an image by considering each row and column of an image as a 1D signal and then applying these filters. But we can also define 2D discrete convolution, which is simply an extension of the 1D case. Given two discrete-time signals, $u[n]$ and $v[n]$, their convolution can be defined as:

$$w(i, j) = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} u(i, j) \cdot v(i - m, j - n).$$

This kind of operation is extensively used in the field of digital image processing wherein the 2D matrix representing the image will be convoluted with a comparatively smaller matrix called 2D kernel. This concept can be extended to multi-dimensional signals, leading to definition of multi-dimensional convolution.

MATLAB commands: As we know that `conv()` is used to implement 1-D filters or we can also use `firfilt` for the same purpose, and to implement two-dimensional (2-D) filters MATLAB command `conv2()` can be used. The 2-D filtering operation actually consists of 1-D filters applied to all the rows of the image and then to all the columns.

Pre-lab

- Before starting this lab we need to download two toolboxes that are required for this lab (for those working online no need to download any toolbox):
 1. Image Processing Toolbox
 2. Signal Processing Toolbox
- Download the folder of Lab8 present in files section and copy paste all the files in MATLAB working directory.

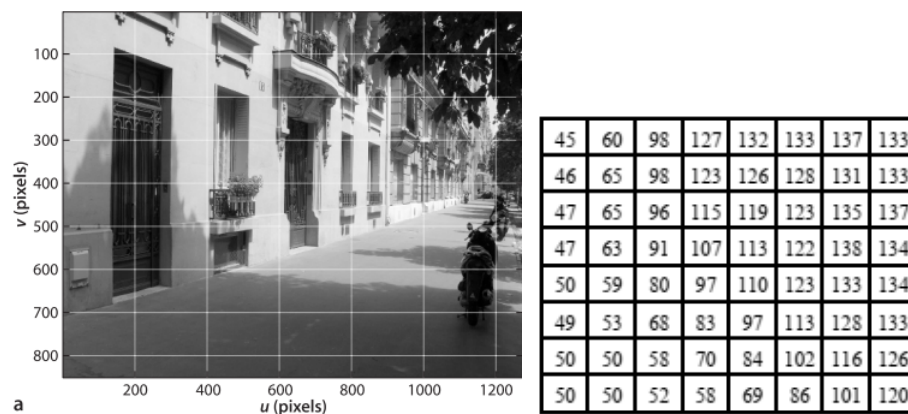
Expectation: If this toolbox is not downloaded earlier you will not be able to complete the lab.

3 Images and Image Processing

In the last lab, we discovered that the 5-point moving average filter is a low-pass filter and has the effect of smoothing a signal. It's reasonable to assume that if the same filter were applied to an image, we shall observe the image being smoothed or the image will be blurred.

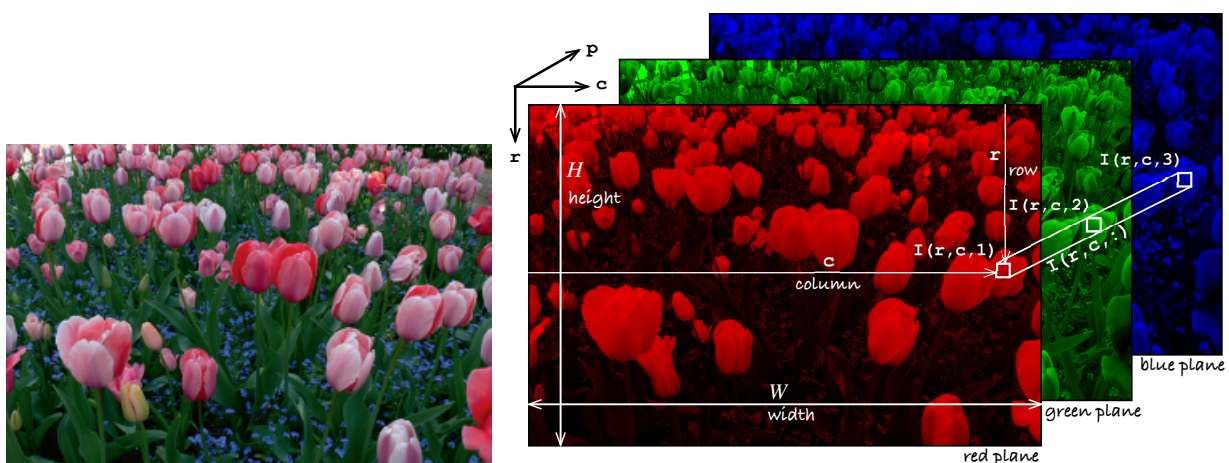
3.1 Representation of an image in MATLAB

In MATLAB a grayscale image of size $W \times H$ is represented as a matrix of size $W \times H$, where W is the width of the image and H is the height of the image. The values in the matrix are



the intensities of the pixels at the corresponding locations. Typically, intensity is expressed as an integer between 0 and 255, with 0 being a black pixel and 255 is the brightest pixel. The location $I(r, c)$ of a matrix I refers to the r th row and c th column pixel. The origin (0,0) is in top left corner of the image.

A color image is a combination of three 2-D arrays - one for red intensities, one for green, and one for blue.



3.2 Image Blurring

Task 1 - Spatial blurring

In this task, we will use 2-D convolution to blur the image of a flower by convolving it with a 2-D version of a 5-point moving averaging filter.

1. Read an image using `imread` and display it using `imshow`.

```
%Read image, visualize it, and display its size and class.  
I = imread("flower.jpg");  
% Convert to double precision  
I = im2double(I);  
% Display  
figure,  
imshow(I)  
whos I
```

Note: Since this is a color image, notice that `I` is a $W \times H \times 3$ array in the workspace. In MATLAB, double matrices representing images are expected to have values in the range $[0,1]$.

2. We will perform three separate 2-D convolutions to blur each of the three color planes, but to do that we have to separate the planes.

```
% Split the image into the 3 component color intensity planes or matrices.  
[R, G, B] = imsplit(I);  
whos R
```

3. `R`, `G`, and `B` are now 2-D signals. Let's create a 2-D filter to blur them. Recall that the impulse response of 1D 5-point moving average filter was simply $\frac{1}{5} [1 \ 1 \ 1 \ 1 \ 1]$. Analogous to this, the impulse response of a 2D 5×5 moving average filter is created below.

```
% Create a 2-D convolution filter or kernel for averaging or smoothing  
W = 5;  
h = ones(W)./W.^2
```

Task 1 - continued

4. Use `conv2` to convolve this averaging impulse response with each of the color planes - R, G, and B. One of them has been pre-filled; Complete the other two similarly. Use the "same" shape to ensure that the output of convolution is of the same size as the input image.

```
Convolve R with h to blur the image
Ravg = conv2(R,h,"same");
% Convolve G with h to blur the image
Gavg = _____ %Write your code here
% Convolve B with h to blur the image
Bavg = _____ %Write your code here
% Concatenate the three matrices to put together the color image
Iavg = cat(3,Ravg,Gavg,Bavg);
whos Iavg
imshow(Iavg,[])
```

- Increase the filter size W . What is the effect of increasing the size of the filter?
- Look closely at the boundaries of the output image. What happens to the border pixels? Why?

You may attempt the above two questions later on and move to the next task.

3.3 Edge Detection

We'll briefly take a break from processing images to introduce a new filter: first difference filter. This filter has a very simple impulse response of, which is:

$$h[n] = \delta[n] - \delta[n - 1]. \quad (1)$$

In the following task, you'll further your understanding of this filter by applying it to a 1D signal.

Task 2: First difference filter

Apply first difference filter to the signal, $x[n] = \text{double}((\sin(0.5 \cdot n) + 0.2) < 0)$, for $n=0:50$.

The impulse response $h[n]$ of the first difference filter is given above in equation 1.

Notice the points at which the output is non-zero and find out how these are related to the transition points of the original signal. Comment on it.

We introduced the first difference filter because it can detect edges in an image. It does so by identifying the transition in values, as it is reasonable to assume that the intensity of pixels will vary at an edge. In the following task you will apply first difference filter to detect edges in the image below.

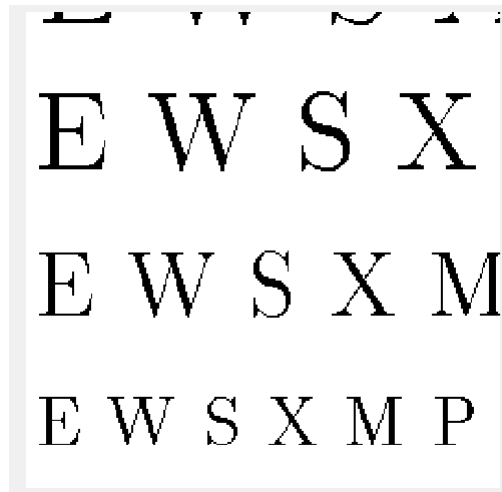


Figure 2: Image echart

Task 3: Edge Detection

- Load in the matrix `echart.mat`, available on LMS. This can be accomplished by using MATLAB's `load` function. This is a 257×256 size image.
 - Display the image. You can display it in MATLAB using the `imshow` function.
 - You can display a portion of this image by selecting rows and column from the matrix. Try `imshow(echart(1:65,:))`.
 - Now apply the first difference filter on one of the rows of this image. Choose a row with a lot of transitions between black and white, such as row 65, 147, or 221. For example, choosing row 65, `echart(65,:)` and now implement a first difference filter on it.
1. Plot both the input and output signals on the same plot with `stem`. Give proper title and axis labelling.
 2. What do the zeros (0) and ones (255) represent in the input signal? What do the non-zero values represent in the output signal $y[n]$?

4 Bar Code Detection and Decoding

A 12-digit bar code consists of alternating black and white bars; the white bars appear to be spaces. The UPC (Universal Product Code) uses widths of the bars to encode numbers. There are four widths that are integer multiples of the thinnest black bar, or thinnest white space. Thus, we define a 3 wide black bar as three times as wide as the thinnest black bar; likewise, for 2-wide and 4-wide bars—whether black or white. Look at any bar code, and you should be able to identify the four widths. Each number from 0 to 9 is encoded with a quadruplet. Here is the encoding of the digits 0-9:

For example, the code for the number “5” is 1-2-3-1, meaning it could be a one-unit wide white

0 = 3-2-1-1	5 = 1-2-3-1
1 = 2-2-2-1	6 = 1-1-1-4
2 = 2-1-2-2	7 = 1-3-1-2
3 = 1-4-1-1	8 = 1-2-1-3
4 = 1-1-3-2	9 = 3-1-1-2

Figure 3: Encoded Barcode

space, followed by a 2-wide black bar, followed by a 3-wide white space, and finally a 1-wide black bar (or inverted: 1-wide black, 2-wide white, 3-wide black, and 1-wide white).

The UPC (Universal Product Code) consists of twelve digits delimited on each end by 1-1-1 (black-white-black), and separated in the middle (between the sixth and seventh digit) by white-black-white-black-white (1-1-1-1-1). Thus the entire UPC must have 30 black bars and 29 white bars for a total of 59.

Furthermore, note that the encoding for each digit always adds up to seven so the total width of the bar code is always the same. In terms of the unit width where the thinnest bars have width one, it should be easy to determine that the total width of the UPC bar code is 95 units.



Figure 4: A sample Barcode

Task 4: Bar Code detection and decoding

In this task, we are now going to develop a system in MATLAB for detecting a bar code from an image. You are provided a function to decode the barcode, but you need to process the image to find out the widths of these bars, which is the input to this decode function.

1. Read the image HP110v3.png into MATLAB using `imread` function. Extract one row from the middle of the image to define a 1-D signal $x[n]$ for further processing.
Hint: Number of rows in I can be seen in workspace, or we also have `size` command for that.
2. Filter this signal using a first difference filter, as described above, and call the output $y[n]$. Plot both $x[n]$ and $y[n]$ on the same plot and compare.
3. The spikes with higher magnitude in the output signal $y[n]$ represent the edges in the image where transitions between black and white happen. You'll notice that the magnitudes are different; Why is that?

If you need an indicator for the edges, then you must define an additional system whose output is 1 (true) at the exact edge location, and 0 (false) otherwise. For example:

$$d[n] = \begin{cases} \text{Edge true or 1} & \text{if } |y[n]| \geq \tau \\ \text{Edge false or 0} & \text{if } |y[n]| < \tau \end{cases} \quad (2)$$

Here, τ is the value of the threshold. Implement this thresholding system, which takes the absolute value of $y[n]$ as an input and returns the indicator signal $d[n]$ as an output.

Hint: Observe the above plot, and select the threshold value according to the magnitude of the higher peaks (rounded value is preferred).

4. We will now use MATLAB's `find` function to produce a location signal $l[n]$, which contains indices or locations in the image where edges occur (i.e. indices of non-zero values) in $d[n]$.
Explore `find` function and write its purpose in comments.
5. Apply first-difference filter to the location signal $l[n]$ to obtain $\Delta[n]$ (`delta[n]`). This signal of differences should be the widths of the bars.

Task 5: continues

6. We have to deal with a few problems now. Ideally, we should have four widths; if the basic width is θ then the four widths are $\theta, 2\theta, 3\theta$, and 4θ . If the width of the thinnest bar is not an integer number of pixels then we will obtain a range of different widths. For example, when the basic width is 3.5, we would expect the plot of $\Delta[n]$ to jump between 3 and 4 for 1-wide bars. Therefore, we need to estimate the basic width θ and use it to fix $\Delta[n]$. Obtain the basic width by utilizing the following two facts:
 - Every valid 12-digit bar code has 59 bars
 - Total width of a 12-digit bar code is always 95θ .
7. The other problem is that the length of $\Delta[n]$ (representing number of bars) will be greater than 59 because of the gaps and other artifacts. We will leave this problem for now by choosing appropriate section of $\Delta[n]$, to estimate θ using the above procedure. Fix $\Delta[n]$ by dividing by θ and rounding. The resulting widths should just be integers 1, 2, 3 or 4.

Hint: We only need 59 bars, so we will have to ignore some starting and ending bars.
8. Now you have processed the signal and it is ready for decoding. Pass the appropriate section of the fixed $\Delta[n]$ to the function `decodeUPC`. The function takes a vector of 59 integers as input. The function will return the decoded bar code (On command window). Show results.



Assessment Rubric
Lab 7: Convolution-II

Name:	Student ID:
--------------	--------------------

Marks distribution:

	Tasks	LR2 Code	LR5 Results	LR9 Report	AR4 Report Submission	AR7 Comments
In-Lab	Task 1		/8	/04	/02	
	Task 2	/8	/4			/2
	Task 3	/8	/4			/2
	Task 4	/12	/12			/4
Total =70		/28	/28	/04	/02	/8
CLO Mapped		CLO2	CLO2	CLO2	CLO3	CLO3

CLO – (LDL)	Total Points	Points Obtained
CLO2 – (Psy-4)	56	
CLO3 – (Aff-3)	14	
Total	70	

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.



Lab Evaluation Assessment Rubric

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3: Good Points 3	Level 4: Exemplary Points 4
LR2	Program/Code/ Simulation Model/ Network Model	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR3	Troubleshooting	Unable to identify the fault/minimal effort show in troubleshooting.	Able to identify the fault but unable to remove it.	Able to identify the fault but partially removes it.	Able to identify the fault and takes necessary steps and actions to correct it.
LR4	Data Collection	Measurements are incomplete, inaccurate and imprecise. Observations are incomplete or not included. Symbols, units and significant figures are not included.	Measurements are somewhat inaccurate and imprecise. Observations are incomplete or vague. Major errors are there in using symbols, units and significant digits.	Measurements are mostly accurate. Observations are generally complete. Minor errors are present in using symbols, units and significant digits.	Measurements are both accurate and precise. Data collection is systematic. Observations are very thorough and include appropriate symbols, units and significant digits and task completed in due time.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
LR6	Calculations	Formulae used and/or calculations are incorrect. Units are not mentioned with results.	Formulae used are correct but there are few mistakes in calculation which lead to incorrect results.	Formulae used and end results are correct. Complete working steps are not shown. Proper units are not mentioned with results.	Formulae used, end results and all calculations are correct with all the intermediate steps clearly shown. Units are mentioned with results.
LR9	Report	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included. Report is submitted after due date.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables.
LR11	Design	Proposed design is unsubstantiated or does not satisfy most of given constraints. The breakdown of system into features is incomplete and still at lower resolution.	Justification for proposed design is weak, and it only satisfies some constraints. The breakdown of system is missing some features and resolution is not appropriate at some places.	Proposed design is substantiated, preferably through proper analysis, and satisfies most given constraints. The breakdown of system into features seems complete, but resolution is not appropriate at some places.	Proposed design is substantiated, preferably through proper analysis, and satisfies all given constraints. The breakdown of system into features seems complete and at appropriate resolution, given students' level.
AR4	*Report Submission	Not accepted after 1 week.	Late submission after 2 days and within 1 week.	Late submission after the lab timing and within 2 days of the due date.	Timely submission of the report and in the lab time.
AR7	Report Content/Code comments	Most of the questions are not answered / figures are not labelled/ titles are not mentioned / units are not mentioned. No comments are present in the code.	Some of the questions are answered, figures are labelled, titles are mentioned and units are mentioned. Few comments are stated in the code.	Majority of the questions are answered, figures are labelled, titles are mentioned and units are mentioned. Comments are stated in the code.	All the questions are answered, figures are labelled, titles are mentioned and units are properly mentioned. Proper comments are stated in the code.